

Tesina

Licenciatura en Estadística

Análisis de comentarios en redes sociales con latent Dirichlet allocation



Facultad de Ciencias Económicas y Estadística

Alumna: Alfonsina Badin

Director: Ignacio Evangelista

2025

Índice

1	Agradecimientos	3
2	Resumen	4
3	Introducción	5
4	Objetivos	7
4.1	Objetivo general	7
4.2	Objetivos específicos	7
5	Metodología	8
5.1	Estadística Bayesiana	8
5.1.1	Modelos hacia atrás: inferencia bayesiana	8
5.1.2	Modelos hacia adelante: modelo generativo	10
5.1.3	El modelo jerárquico	12
5.1.4	Markov Chain Monte Carlo	13
	Muestreo de Gibbs	14
5.2	Machine learning	15
5.2.1	Aprendizaje supervisado	15
5.2.2	Aprendizaje no supervisado	15
5.3	Procesamiento de Lenguaje Natural (NLP)	16
5.3.1	Recolección de datos	17
	Uso de conjuntos de datos públicos	17
	Obtención de datos de internet	17
	Combinación de fuentes de datos	17
5.3.2	Limpieza de texto	17
	Pasos preliminares	18
	Tokenización	18
	Normalización de Unicode	18
	Eliminación de stopwords	19
	Corrección ortográfica	20
	Corrección ortográfica empleando distancia de Levenshtein-Damerau	21

	Lematización	23
5.3.3	Representación de texto	24
	Representación vectorial	24
	One-Hot Encoding	24
	Bag of words	25
	TF-IDF	26
	Embeddings	27
	Word2Vec	27
	GloVe	28
	BERT	29
5.4	Latent Dirichlet allocation	29
5.4.1	LDA generativo	30
5.4.2	Inferencia con LDA	32
6	Bibliografía	36

1 Agradecimientos

2 Resumen

3 Introducción

A lo largo de los años, las redes sociales se han convertido en el medio de comunicación más utilizado. Diariamente, los usuarios ingresan a ellas para conversar con otras personas, dar a conocer sus puntos de vista, compartir experiencias, informarse sobre las últimas noticias y entretenerse.

En el año 2020, a raíz de la pandemia mundial por COVID-19, hubo un tipo de contenido que se popularizó con gran velocidad: el *streaming*. El *streaming* es una combinación de radio y televisión, que se transmite en vivo en una plataforma gratuita (YouTube o Twitch) y luego es publicado *On-Demand* para que el usuario pueda verlo fuera de su horario habitual si así lo desea. Por lo general el formato consta de un panel de conductores que charlan de tópicos comunes para la audiencia, generando debates interesantes con los que los usuarios se pueden llegar a identificar. Dentro de cada canal de *streaming* se transmiten distintos programas que tienen un perfil particular y objetivos diferentes ya que su contenido puede variar en entretenimiento, humor, chimentos, noticias, etc.

Su popularidad se debe a que es un contenido gratuito, cercano y que genera una **comunidad** entre los conductores y sus oyentes, quienes no sólo interactúan en vivo a través de un *chat* sino que dejan sus comentarios en otras redes sociales: Instagram, TikTok, YouTube, entre otros.

Al publicar los programas *On-Demand*, los canales de *streaming* acceden a un recurso valioso que refleja la opinión de sus oyentes: los comentarios de YouTube. El análisis de estos textos puede resultar en conclusiones interesantes para los canales ya que permite un entendimiento de los sentimientos que tiene su comunidad con respecto a cada contenido.

Para llevar a cabo este análisis, el presente informe utiliza **latent Dirichlet allocation (LDA)**, una técnica de aprendizaje automático no supervisado que clasifica automáticamente los textos en diferentes categorías o temas según las características del corpus. Esta metodología permite detectar patrones temáticos recurrentes en grandes volúmenes de datos textuales, como los comentarios de YouTube, y extraer información valiosa.

LDA se basa en un modelo probabilístico generativo y es especialmente útil para modelar datos discretos como textos. Es un modelo bayesiano jerárquico de tres niveles, en el cual cada documento (en este caso, cada comentario) se representa como una mezcla de temas, y cada tema se define como una mezcla de palabras. En este sentido, LDA no solo agrupa comentarios en función de temas compartidos, sino que también asigna probabilidades a cada palabra dentro de cada tema, proporcionando una representación explícita y estructurada del contenido del corpus.

Esta capacidad de modelar documentos como combinaciones de múltiples temas lo hace particu-

larmente adecuado para analizar la diversidad temática de los comentarios de YouTube. Por ejemplo, un comentario podría estar relacionado en un 60% con el tema “entretenimiento” y en un 40% con el tema “noticias”, lo que permite identificar la interacción entre los intereses de los usuarios.

Esta tesina tiene por objeto de estudio la técnica latent Dirichlet allocation. Se brinda una introducción al tema, abarcando las definiciones básicas, sus componentes, sus variantes, sus ventajas y limitaciones y sus campos de aplicación. Se incluyen también los conceptos teóricos necesarios para comprender la metodología. Adicionalmente, se presenta la aplicación del modelo en comentarios de redes sociales.

4 Objetivos

4.1 Objetivo general

El objetivo general del presente proyecto es profundizar en el estudio y aplicación del modelo latent Dirichlet allocation (LDA) para la identificación de tópicos o categorías.

4.2 Objetivos específicos

- Comprender las bases del procesamiento de lenguaje natural (NLP por sus siglas en inglés) y las técnicas de representación computacional de texto.
- Mencionar los desafíos y limitaciones que conlleva el ajuste de LDA y propuestas que han surgido para contrarrestar dichos inconvenientes.
- Aplicar LDA en un conjunto comentarios en español (argentino) en la red social YouTube para comprender las opiniones de los oyentes.

5 Metodología

5.1 Estadística Bayesiana

La estadística bayesiana es un enfoque de la estadística que, haciendo uso del teorema de Bayes, actualiza la probabilidad de una hipótesis a medida que se dispone de más datos haciendo una combinación entre observaciones y creencias previas. A diferencia de la estadística frecuentista, que interpreta las probabilidades como frecuencias relativas de eventos en un gran número de ensayos, la estadística bayesiana trata las probabilidades como grados de creencia o confianza.

Teorema de Bayes: Sea θ una hipótesis o conjetura que se tiene sobre el fenómeno en estudio y E evidencia o nueva información al respecto, se puede actualizar el grado de incertidumbre (o creencia) de esa hipótesis teniendo en cuenta la nueva evidencia como

$$p(\theta|E) = \frac{p(E|\theta)p(\theta)}{p(E)}$$

La **probabilidad a priori** es la creencia inicial sobre la hipótesis, se realiza antes de observar los datos para luego actualizarla mediante la probabilidad condicional (regla de Bayes), obteniendo así la **probabilidad posterior**. Esta probabilidad posterior se convierte en la nueva creencia actualizada después de considerar la evidencia proporcionada por los datos.

En el enfoque bayesiano entonces, cada conjunto de nuevos datos permite actualizar la probabilidad posterior, lo que lo hace especialmente útil en contextos donde se obtiene información de manera continua. Por ejemplo, la puntuación de un hotel se puede actualizar con cada opinión obtenida mediante un modelo bayesiano (Figura 1).

Un modelo estadístico en este contexto, integra información previa y actual y se pueden definir los modelos hacia atrás (inferencia) o los modelos hacia adelante (generativos) para el análisis de un fenómeno.

5.1.1 Modelos hacia atrás: inferencia bayesiana

Los modelos hacia atrás se enfocan en deducir información sobre parámetros desconocidos a partir de datos observados. Este tipo de modelo es útil cuando el interés principal está en obtener conclusiones sobre aspectos no observables de una población. En este caso, la probabilidad posterior es la clave, ya que refleja el grado de creencia actualizado en los parámetros dados los datos.

Continuando con el ejemplo del hotel, siendo X la cantidad de veces en las que una persona puntúa a un hotel con un calificación de i y p_i la probabilidad de hacerlo, se tiene que



Figura 1: Ejemplo de modelo bayesiano

$X \sim \text{Multinomial}^{(1)}(p_1, p_2, p_3, p_4, p_5)$. Estas probabilidades pueden ser modeladas utilizando la distribución *Dirichlet* ya que permite capturar la incertidumbre sobre las probabilidades de cada categoría: modelo *Dirichlet* – *Multinomial*.

Suponiendo inicialmente que no se tiene preferencias sobre la cantidad de estrellas, se utiliza un prior *Dirichlet* uniforme:

$$p = (p_1, p_2, p_3, p_4, p_5) \sim \text{Dirichlet}(\alpha_1, \alpha_2, \alpha_3, \alpha_4, \alpha_5)$$

donde $\alpha_i = 1 \forall i = 1, 2, 3, 4, 5$. Luego, para cada hotel, se puede calcular la distribución posterior del vector de parámetros p utilizando la fórmula de la distribución *Dirichlet* posterior e información existente del puntaje de los hoteles.

$$\text{Dirichlet}_{\text{Hotel1}} \sim (1215, 418, 68, 9, 8)$$

$$\text{Dirichlet}_{\text{Hotel2}} \sim (856, 819, 260, 89, 57)$$

Con estas distribuciones se realizan 1000 simulaciones de $\mathbf{p}$ que darán cuenta de la distribución posterior de estas probabilidades. En la Figura 2 se puede observar la densidad de cada elemento de $\mathbf{p}$ con cada posterior.

Ambos hoteles tienen baja probabilidad de tener calificaciones malas (y con mucha credibilidad), y en la calificación más alta se observa que el Hotel 1 es superadora ¿Qué sucedería si por una falla en la

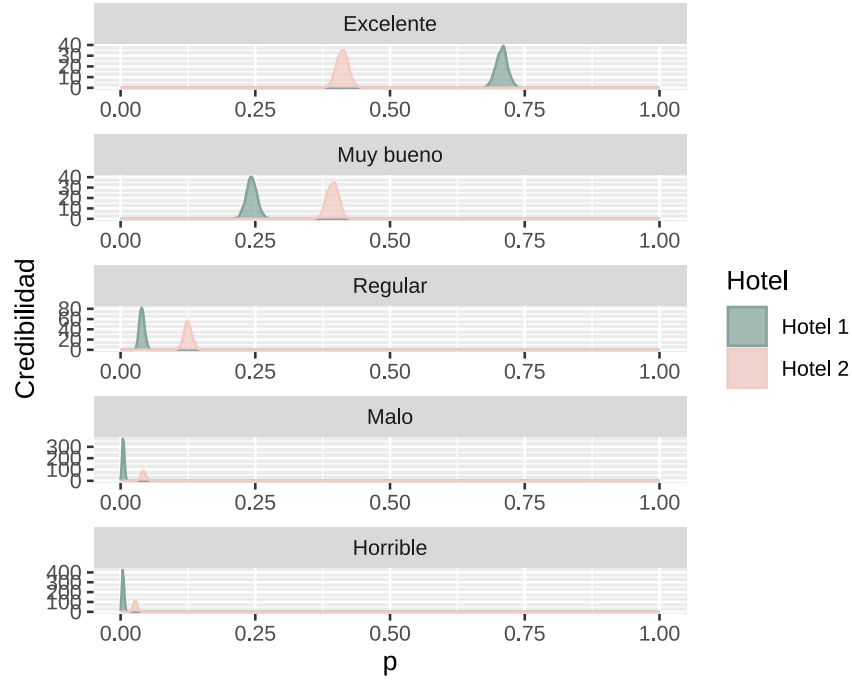


Figura 2: Densidad de los puntajes para cada hotel

cocina se agregan 100 puntajes de ‘Horrible’ para este hotel? Su distribución quedaría $Dirichlet_{Hotel1} \sim (1215, 418, 68, 9, 108)$ y su nuevo posterior se modifica notoriamente, tal como se observa en la Figura 3.

La inferencia bayesiana intenta asignar una cierta credibilidad a cada resultado posible y actualizarla a medida que se recolecta información.

5.1.2 Modelos hacia adelante: modelo generativo

Los modelos hacia adelante parten de suposiciones sobre los parámetros o las distribuciones que generan los datos y buscan modelar cómo estos datos podrían haberse producido. Este enfoque es útil para simular datos y predecir resultados futuros basados en los modelos probabilísticos definidos. En la estadística bayesiana, este proceso incluye tanto la generación de datos simulados como la comparación con datos observados para refinar modelos y creencias.

Continuando con el ejemplo anterior con una perspectiva generativa, se podría buscar el puntaje promedio que le espera al Hotel 1 utilizando la última distribución posterior. Para ello se realizan 1000 simulaciones de $\mathbf{p}$ usando la distribución $Dirichlet_{Hotel1} \sim (1215, 418, 68, 9, 108)$, así se obtiene un conjunto de 1000 vectores que le asignan diferentes probabilidades a los puntajes.

Con este conocimiento sobre el comportamiento de la clasificación del Hotel 1, se podría esperar con gran credibilidad que el hotel obtenga a futuro una calificación “Horrible” o “Malo”. Sin dudas las últimas

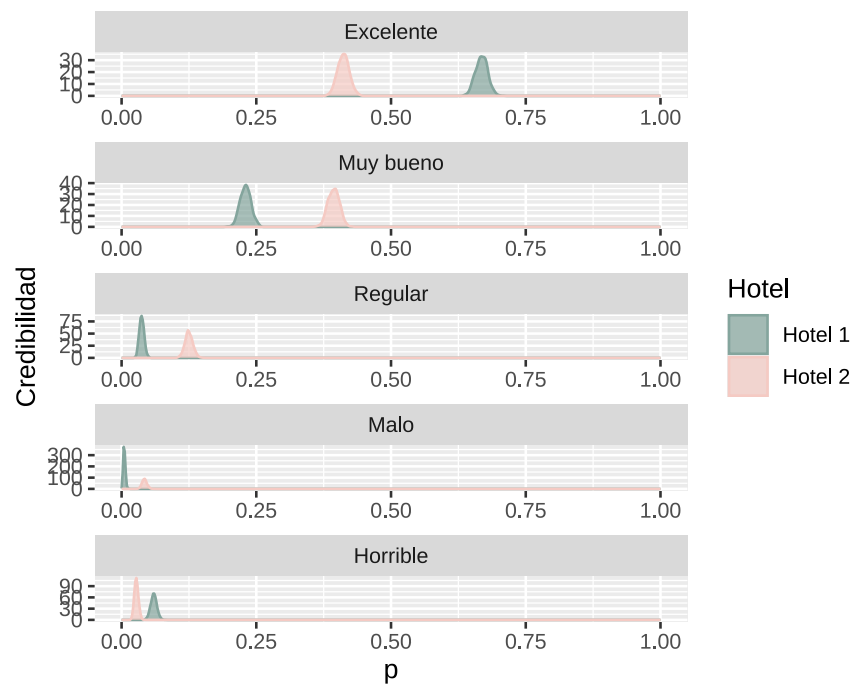


Figura 3: Densidad de los puntajes para cada hotel con los nuevos puntajes del Hotel 1

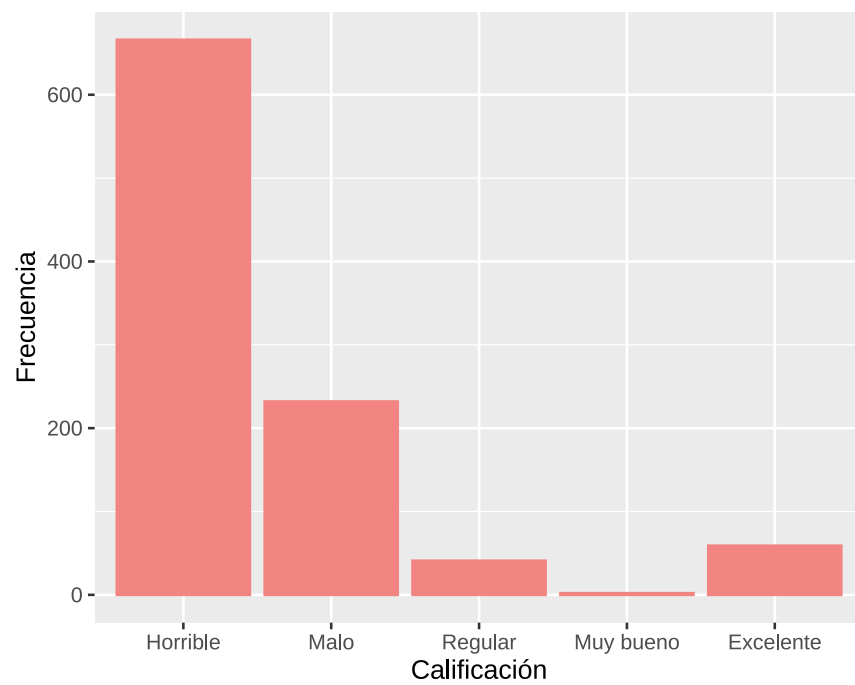


Figura 4: Frecuencia esperada para cada clasificación

100 calificaciones malas tuvieron gran impacto en la reputación del hotel.

Este modelo usa la probabilidad a priori para simular cómo los datos podrían verse antes de tener datos reales. Es útil para prever el comportamiento de los datos bajo ciertas suposiciones.

5.1.3 El modelo jerárquico

Un modelo jerárquico es una expresión que relaciona un conjunto de parámetros a través de la credibilidad de sus estimaciones, de forma que la estimación de un parámetro depende de la estimación de otro(s) parámetro(s).

Por ejemplo, al estudiar la probabilidad de recuperación de una operación cardíaca con un conjunto de datos que recopila información de pacientes en distintos hospitales, diferentes ciudades y que han sido operados por diversos equipos, se debería hacer uso de un modelo jerárquico. Cada equipo quirúrgico tiene un cierto número de recuperaciones dentro de una cantidad de cirugías, entonces la probabilidad de recuperación de cada uno θ_e depende de un parámetro ω_h que describe la probabilidad típica de recuperación para el hospital, que a su vez depende de un parámetro α_c que describe la probabilidad a nivel ciudad.

Lo que hace que el modelado jerárquico sea tan efectivo es que la estimación de cada parámetro individual se ve influenciada simultáneamente por los datos de todos los demás individuos. Esto ocurre porque todos los individuos informan a los parámetros de nivel superior, los cuales a su vez limitan todos los parámetros individuales. Estas dependencias estructurales entre parámetros proporcionan estimaciones más informadas de todos los parámetros.

Los parámetros que están en distintos niveles de la jerarquía coexisten en un espacio paramétrico conjunto, independientemente del nivel al que pertenezcan, y es donde interviene la regla de Bayes.

Formalmente, sean θ y ω dos parámetros, la regla de Bayes aplicada al espacio conjunto implica:

$$p(\theta, \omega | D) = \frac{p(D | \theta, \omega) p(\theta, \omega)}{p(D)}$$
$$p(\theta, \omega | D) \propto p(D | \theta, \omega) p(\theta, \omega)$$

Lo que resulta particular de los modelos jerárquicos es que los términos del lado derecho pueden factorizarse siguiendo una cadena estructurada de dependencias. En el caso más simple:

$$p(\theta, \omega | D) \propto p(D | \theta) p(\theta | \omega) p(\omega)$$

Esta refactorización refuerza que los datos dependen únicamente del parámetro θ y que este depende del parámetro de nivel superior ω , mientras que ω es independiente del resto de los parámetros.

5.1.4 Markov Chain Monte Carlo

El objetivo principal de la inferencia bayesiana es obtener una representación acertada de la distribución posterior. Una de las maneras de hacerlo es tomar una gran muestra de puntos representativos del *posterior*, pero, por lo general, no es posible acceder de una forma explícita. Entonces se utilizan métodos de *Markov Chain Monte Carlo* (MCMC) para aproximar dicha distribución mediante el uso de cadenas de Markov.

Una cadena de Markov es un proceso estocástico que describe una secuencia de estados donde la probabilidad de pasar al estado siguiente depende únicamente del estado actual y no del historial previo; esta propiedad se conoce como memoria corta. En el contexto de los algoritmos MCMC, las cadenas de Markov se utilizan para generar muestras dependientes que, tras un período de convergencia, representan fielmente la distribución posterior que se desea aproximar.

Uno de los algoritmos fundamentales de MCMC es el algoritmo de Metrópolis. Su idea central es que, aún sin conocer la forma exacta de la distribución objetivo, se puede construir una cadena de Markov que “visite” distintas regiones del espacio paramétrico con una frecuencia proporcional a la probabilidad posterior.

Imaginando una avenida con muchos hoteles, cada uno con una calificación promedio (de 1 a 5 estrellas) dada por los huéspedes. Un viajero desea pasar una cantidad de noches en hoteles proporcionales a cuán bien valorados están, pero no conoce de antemano qué calificación tiene cada hotel ni cuántos hoteles hay en total.

Entonces, el viajero procede a:

1. Elegir al azar un hotel vecino que quiera visitar.
2. Si la calificación del hotel propuesto es mayor a la del hotel actual, acepta el cambio y se muda.
3. Si la calificación del hotel propuesto es menor, decide mudarse si el cociente entre ambas calificaciones es mayor que un número aleatorio entre 0 y 1. De esta manera, existe la posibilidad de mudarse a hoteles peores, aunque sea una noche, sin perder de vista el objetivo inicial.
4. Si no se muda, permanece en el hotel actual y vuelve a proponer un nuevo movimiento al día siguiente.

Después de suficientes noches, el viajero terminará pasando más tiempo en los hoteles con mejores calificaciones, aunque nunca haya conocido el mapa completo de hoteles ni sus valores globales.

El algoritmo de Metropolis funciona de manera análoga:

- la calle con hoteles representa el espacio paramétrico,
- la calificación de cada hotel es el valor de la densidad posterior en ese punto,

- el movimiento probabilístico entre hoteles replica el criterio de aceptación del algoritmo.

Así, incluso sin conocer la forma completa de la distribución posterior, el algoritmo garantiza que puntos con mayor probabilidad posterior serán visitados con mayor frecuencia, permitiendo aproximar la distribución a posteriori mediante las muestras generadas.

Muestreo de Gibbs

Aunque el algoritmo de Metropolis resulta intuitivo y conceptualmente simple, su aplicación directa a modelos con muchos parámetros presenta limitaciones importantes. En espacios paramétricos de alta dimensión, proponer un nuevo valor para todo el vector de parámetros a la vez suele conducir a propuestas muy alejadas del estado actual, lo que provoca tasas de aceptación extremadamente bajas. Alternativamente, propuestas demasiado pequeñas generan cadenas que avanzan muy lentamente. En ambos casos, la eficiencia del muestreo disminuye drásticamente y la cadena puede tardar mucho tiempo en explorar adecuadamente la distribución posterior.

Una estrategia natural para mitigar estos problemas consiste en actualizar un parámetro a la vez, manteniendo los demás fijos. Esta idea convierte una propuesta multidimensional en una secuencia de propuestas unidimensionales mucho más manejables. Si, además, el modelo es jerárquico y utiliza distribuciones conjugadas, es posible obtener las distribuciones condicionales completas de cada parámetro: distribuciones exactas que describen cómo debería comportarse un parámetro dado el valor actual de todos los demás.

El algoritmo *Gibbs Sampling* surge como una alternativa eficiente al algoritmo de Metropolis en modelos con múltiples parámetros y estructura jerárquica. En lugar de generar propuestas arbitrarias y luego decidir si aceptarlas, Gibbs actualiza cada parámetro muestreándolo directamente de su distribución condicional completa, es decir, la distribución del parámetro dado el valor actual de todos los demás. Debido a que estas condicionales provienen de la propia estructura del modelo, cada actualización es automáticamente coherente con la distribución objetivo y, por lo tanto, la probabilidad de aceptación es siempre uno. En este sentido, Gibbs puede verse como un caso particular del algoritmo de Metropolis en el que las propuestas nunca se rechazan.

En términos formales, si el vector de parámetros es $\theta = (\theta_1, \dots, \theta_p)$, el algoritmo consiste en actualizar secuencialmente cada componente muestreando las distribuciones:

$$\theta_1^{t+1} \sim p(\theta_1 | \theta_2^{(t)}, \theta_3^{(t)}, \dots, D),$$

$$\theta_2^{t+1} \sim p(\theta_2 | \theta_1^{(t)}, \theta_3^{(t)}, \dots, D)$$

Tras suficientes iteraciones, esta secuencia de actualizaciones genera una cadena de Markov cuya

distribución estacionaria coincide con la distribución posterior conjunta, permitiendo aproximarla mediante las muestras de la cadena.

5.2 Machine learning

El Aprendizaje Automático (*machine learning*, ML) es una rama de la inteligencia artificial que permite a las computadoras aprender patrones y tomar decisiones a partir de datos, sin ser programadas explícitamente para cada tarea. Es especialmente útil en problemas complejos, donde hay muchas reglas por seguir y/o entornos inconsistentes ya que los modelos de ML se adaptan fácilmente a nuevos datos.

Un ejemplo utilizado en lo cotidiano es el filtro de *spam* de la casilla de correo electrónico. Con algunos ejemplos de correos *spam* y correos legítimos o *ham* (datos de entrenamiento), el código detrás del filtro logra aprender la naturaleza de cada uno de estos elementos, logrando así clasificar nuevos casos. Los modelos de ML se evalúan con métricas de *performance* que se miden con datos de prueba.

Los sistemas de *machine learning* se pueden clasificar en función de la cantidad de supervisión que reciben en el entrenamiento.

5.2.1 Aprendizaje supervisado

En el aprendizaje supervisado, los datos de entrenamiento que se usan como entrada al algoritmo incluyen los resultados deseados (*labels*). Por ejemplo, si el filtro de *spam* recibe correos con la etiqueta *spam* y *ham* en el entrenamiento, se está hablando de un aprendizaje supervisado.

Entre las técnicas más importantes del aprendizaje supervisado se encuentran *k-nearest neighbors* (KNN), regresión lineal, regresión logística, *support vector machine* (SVM), árboles de decisión, *random forest*, redes neuronales, etc.

5.2.2 Aprendizaje no supervisado

En el aprendizaje no supervisado, los datos de entrenamiento no cuentan con etiquetas, se dice que el sistema intenta aprender sin profesor. Por ejemplo, si se quisiera agrupar a clientes de un negocio según su consumo, el algoritmo detecta las similitudes y diferencias entre ellos y define un agrupamiento que será evaluado con las métricas de *performance* y ajustado según su resultado.

5.3 Procesamiento de Lenguaje Natural (NLP)

El texto, tal como lo leemos y comprendemos los humanos, es rico en significado pero carece de una estructura numérica, que es la base del procesamiento en los algoritmos de aprendizaje automático. El procesamiento de lenguaje natural es un área de investigación dentro de la informática y la inteligencia artificial (IA) que se ocupa de procesar el lenguaje humano. Este procesamiento generalmente implica traducir el lenguaje natural en datos (números) que una computadora puede usar para aprender sobre el mundo.

El proceso abarca tareas como la recolección, limpieza, normalización y tokenización de texto, estas últimas dos son las responsables de que la información textual pueda ser estructurada de manera que los modelos la interpreten y analicen eficazmente. En la Figura 5 (Vajjala et al., 2020) se pueden observar los pasos que comprende el proceso.

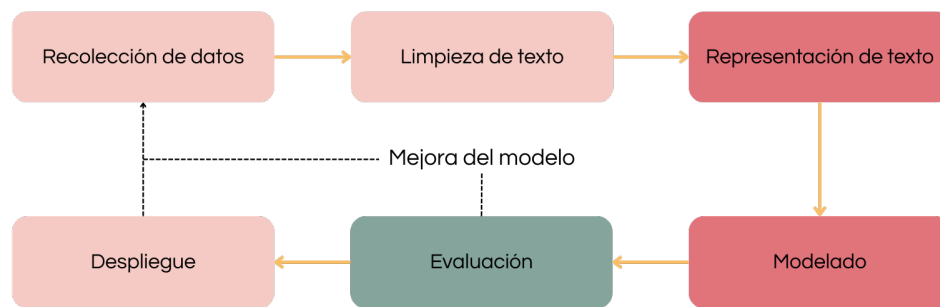


Figura 5: Etapas del NLP

Aunque no sea evidente, el procesamiento de lenguaje natural (NLP) aparece en el día a día de las personas. Tiene una amplia variedad de aplicaciones prácticas, lo que lo convierte en una herramienta esencial en diversos campos, por ejemplo:

- mejoras en motores de búsqueda web,
- corrección ortográfica durante la búsqueda,
- revisión gramatical y ortográfica,
- desarrollo de chatbots y asistentes virtuales,
- generación de índices y tablas de contenido,
- filtrado de spam en correo electrónico,
- personalización en campañas de marketing.

Este amplio rango de usos subraya su importancia como puente entre los lenguajes humanos y las capacidades computacionales.

5.3.1 Recolección de datos

El primer paso esencial en cualquier proyecto de procesamiento de lenguaje natural es la recolección de datos. Esta etapa determina la calidad y relevancia de los resultados que se obtendrán al aplicar técnicas avanzadas como latent Dirichlet allocation (LDA). Existen diferentes enfoques para recopilar datos en NLP, dependiendo de los objetivos del análisis y la disponibilidad de recursos. A continuación, se describen las principales estrategias de recolección de datos:

Uso de conjuntos de datos públicos

Una opción inicial es buscar conjuntos de datos públicos que sean relevantes para la tarea específica. Si el conjunto de datos es adecuado, se puede proceder directamente a construir y evaluar un modelo. Sin embargo, cuando no se encuentra un dataset que cumpla los requisitos, se debe considerar generar datos personalizados.

Obtención de datos de internet

Otra estrategia consiste en identificar fuentes relevantes de datos en internet, como foros de consumidores o plataformas de discusión donde se publiquen consultas. Estos datos pueden ser extraídos automáticamente a través de técnicas de *web scraping* y, posteriormente, etiquetados manualmente por anotadores humanos.

Combinación de fuentes de datos

En la práctica, los conjuntos de datos suelen provenir de fuentes heterogéneas, como *datasets* públicos, datos etiquetados manualmente y datos de internet, al mismo tiempo. Esta combinación es especialmente útil para construir modelos en etapas iniciales, cuando los datos específicos para un escenario particular son limitados.

En esta tesina se optó por hacer uso de una *API* de Google para obtener los datos de entrada para el análisis, este proceso será detallado en la sección ??.

Esta etapa es crucial para garantizar la disponibilidad de datos limpios y útiles, que son la base para las siguientes etapas de procesamiento y modelado. Una vez que se cuenta con los datos recolectados, se procede al siguiente paso: la limpieza y el preprocesamiento de texto.

5.3.2 Limpieza de texto

La extracción y limpieza de texto se refiere al proceso de extraer de los datos de entrada el texto sin procesar, eliminando toda la información no textual, como marcas, metadatos, etc., y convirtiéndolo al formato de codificación requerido. La extracción de texto es un paso estándar en el manejo de datos y, usualmente, no emplea técnicas específicas de NLP. Sin embargo, es un paso importante que tiene implicaciones para todos los demás aspectos. Además, también puede ser la parte más demandante en términos de tiempo dentro de

un proyecto.

Pasos preliminares

Previo a la limpieza del texto, es necesario llevar a cabo una serie de transformaciones básicas que garanticen la homogeneidad del corpus. En primer lugar, se convierte todo el texto a minúsculas con el fin de evitar que palabras idénticas sean tratadas como distintas debido a diferencias en el uso de mayúsculas, por ejemplo, “Video” y “video”. Otro paso fundamental consiste en la eliminación de signos de puntuación, caracteres especiales y espacios innecesarios, que no aportan información semántica al análisis y pueden generar una dispersión artificial en el vocabulario. En esta etapa también se contempla la reducción de espacios múltiples a un único espacio, asegurando así una identificación más precisa de las palabras.

Finalmente, se corrigen patrones de escritura característicos de entornos informales, como la repetición exagerada de letras para enfatizar una emoción (por ejemplo, “amoo” en lugar de “amo” o “encantaa” en lugar de “encanta”). Este tipo de modificaciones, frecuentes en comentarios de redes sociales, pueden aumentar la longitud del vocabulario y dificultar la identificación de términos representativos. La normalización de estas formas al estándar correspondiente resulta clave para mejorar la coherencia interna del corpus y facilitar los pasos posteriores de análisis.

Tokenización

La tokenización es un paso fundamental en el procesamiento del lenguaje natural que tiene como objetivo transformar el texto libre en una secuencia de unidades discretas que puedan ser procesadas computacionalmente. Consiste en segmentar el texto en unidades mínimas llamadas tokens, cuya definición depende de la granularidad aplicada en el proceso:

- A nivel de palabra: cada token corresponde a una palabra. Ejemplo: “Muy bueno el video” → [“Muy”, “bueno”, “el”, “video”].
- A nivel de sub-palabra: cada token es un fragmento de una palabra. Ejemplo: “jugando” → [“jug”, “ando”].
- A nivel de carácter: cada token es un único carácter (letra, signo de puntuación, número, etc.). Ejemplo: “sol” → [“s”, “o”, “l”].
- A nivel de oración: cada token corresponde a una oración completa. Ejemplo: “Hoy llueve. Mañana saldrá el sol.” → [“Hoy llueve.”, “Mañana saldrá el sol.”].

Normalización de Unicode

Al limpiar texto, especialmente que se ha obtenido a través de Web Scrapping, es posible que se encuentren varios caracteres Unicode, incluidos símbolos, emojis y otros caracteres gráficos, algunos ejemplos se muestran

en la Figura 6 (Vajjala et al., 2020).



Figura 6: Caracteres Unicode y su representación gráfica

Para interpretar estos símbolos no textuales y caracteres especiales se utiliza la normalización de Unicode, que convierte el texto en alguna forma de representación binaria para ser almacenado en una computadora. Existen varios esquemas de codificación, y la codificación predeterminada puede variar según el sistema operativo.

El siguiente fragmento de código en R ejemplifica la limpieza de emojis en texto. Utilizando expresiones regulares, se eliminan todos los caracteres que no pertenezcan al alfabeto en minúsculas.

```
comentarios <- c(
  "Me encantó este video ", "Lo volvería a ver!!! ",
  "Qué aburrido no me gustó", "Muy bueno!!! "
)
comentarios_limpios <- gsub("[^a-záéíóúñ ]", " ", tolower(comentarios))
comentarios_limpios <- stringr::str_squish(comentarios_limpios)
comentarios_limpios
```

```
## [1] "me encantó este video" "lo volvería a ver"
## [3] "qué aburrido no me gustó" "muy bueno"
```

La salida es un conjunto de comentarios donde los emojis y otros símbolos fueron eliminados, manteniendo únicamente el texto en minúsculas y sin caracteres no deseados.

Eliminación de stopwords

Las *stopwords* son palabras funcionales que, si bien cumplen un rol gramatical importante (como preposiciones, artículos, pronombres...), no aportan contenido semántico relevante al análisis del significado global del texto.

El objetivo de esta etapa es reducir el ruido lingüístico y centrar el análisis en las palabras que efectivamente comunican información. Por ejemplo, en frases como "gracias por el dato" o "esto es una joya", las palabras *por*, *el*, *es*, *una* son típicamente consideradas *stopwords*, mientras que *gracias*, *dato*, *joya* son términos de interés.

Para llevar a cabo esta tarea se utilizará el paquete `tm` (*Text mining*) que cuenta con un listado de *stopwords* (Tabla 1) y funciones como `removeWords` que permite eliminarlas de un texto dado.

Tabla 1: Algunas stopwords del paquete

de	la	que	el	en	y	a	los	del	se
las	por	un	para	con	no	una	su	al	lo

```
library(tm)
prueba <- c(
  "gracias por el dato",
  "esto es una joya"
)
removeWords(prueba, stopwords("es"))
```

```
## [1] "gracias  dato" "  joya"
```

Corrección ortográfica

La corrección de errores en la escritura es un aspecto fundamental del procesamiento del lenguaje natural ya que los textos pueden contener diferentes tipos de fallos que afectan su comprensión y coherencia. Los errores en la redacción pueden clasificarse en tres categorías principales (*Moyotl-Hernández, 2016*):

- Errores ortográficos: ocurren cuando una palabra escrita no existe dentro del idioma.
- Errores gramaticales: Se presentan cuando las palabras utilizadas existen en el idioma, pero no son correctas en el contexto de la oración.
- Errores de estilo: se refieren a palabras redundantes, ambiguas o repetidas que afectan la claridad del texto.

En este sentido, un corrector ortográfico tiene como objetivo identificar palabras mal escritas dentro de un texto y sugerir la opción más adecuada a partir de un conjunto de términos válidos en el idioma.

Corrección ortográfica empleando distancia de Levenshtein-Damerau

La distancia de Levenshtein o distancia de edición es una medida utilizada para calcular la similitud entre palabras, se trata de un conteo de operaciones requeridas para convertir una cadena de caracteres (una palabra) en otra. Las operaciones de edición son:

- Intersección de un caracter: `hogr` $\rightarrow$ `hogar` (agregar ‘a’ entre la ‘h’ y la ‘g’).
- Eliminación de un caracter: `argentina` $\rightarrow$ `argentina` (eliminar la ‘g’).
- Sustitución de un caracter por otro: `numero` $\rightarrow$ `número` (reemplazar la ‘u’ por la ‘ú’).

Hay una generalización de esta medida que considera el intercambio de dos caracteres como una operación: la distancia de Levenshtein-Damerau. En este método, aparece:

- Transposición de caracteres: `paelta` $\rightarrow$ `paleta` (intercambia el lugar de la ‘e’ y la ‘l’).

El método de corrección basado en la distancia de Levenshtein-Damerau se fundamenta en la búsqueda de la palabra más similar dentro de un diccionario para corregir una palabra mal escrita. Este procedimiento implica generar todas las posibles transformaciones de la palabra errónea mediante las operaciones de edición. Luego, cada una de estas transformaciones se compara con las palabras existentes en el diccionario, y aquellas que coincidan se agregan a una lista de sugerencias. Finalmente, la mejor corrección será aquella con la menor distancia a la palabra original.

En la Tabla 2 se muestran todas las palabras generadas a partir de la palabra errónea `burri` con una sola operación de edición.

Tabla 2: Posibles transformaciones de la palabra `burri` con distancia de edición uno

Operación	Palabras generadas
Eliminación	<code>urri</code> , <code>brri</code> , <code>buri</code> , <code>bur</code>
Inserción	<code>aburri</code> , <code>bburri</code> , <code>cburri</code> , ..., <code>zburri</code> , <code>burris</code> , <code>burria</code> , ..., <code>burriz</code>
Sustitución	<code>aurri</code> , <code>burrs</code> , <code>murri</code> , ..., <code>burro</code> , <code>burrq</code> , ..., <code>burrz</code>
Transposición	<code>ubrri</code> , <code>bruri</code> , <code>burir</code>

El desafío radica en seleccionar la cantidad de operaciones admitidas para la corrección y cuál de estas opciones es la más apropiada. En el siguiente código se define un diccionario ficticio para este caso y se calcula la distancia L-D haciendo uso del paquete `stringdist` para cada palabra del mismo. En la Tabla 3 se observan los resultados.

```

# Paquetes
library(stringdist)
library(dplyr)

# 1. Token a corregir
token <- "burri"

# 2. Diccionario ficticio
diccionario <- c("burro", "burrito", "barri", "burla",
                 "perro", "aburris", "aburro", "caballo")

# 3. Calculamos distancia DL
resultados <- tibble::tibble(
  candidato = diccionario,
  distancia = stringdist(token, diccionario, method = "dl")
) %>%
  arrange(distancia, candidato)

```

Tabla 3: Posibles correcciones para la palabra 'burri'

Candidato	Distancia	Candidato	Distancia
barri	1	burla	2
burro	1	burrito	2
aburris	2	perro	3
aburro	2	caballo	6

Si bien es fácil identificar que **perro** o **caballo** no son buenas correcciones, observando el resultado de la distancia L-D se llega a la misma conclusión. Para este conjunto de datos y el ejemplo en particular, hay correcciones mejores en función de este cálculo.

En síntesis, el uso de la distancia de Levenshtein-Damerau constituye una herramienta potente para la corrección ortográfica, ya que permite identificar palabras candidatas a partir de criterios formales de similitud. No obstante, los desafíos principales radican en establecer un umbral de tolerancia adecuado y en gestionar la carga computacional asociada. Una tolerancia demasiado estricta puede dejar sin corregir errores evidentes, mientras que una tolerancia demasiado laxa puede introducir correcciones incorrectas. A su vez, el proceso de comparar un término erróneo contra todas las palabras de un diccionario extenso es computacionalmente intensivo. De igual modo, disponer de un diccionario amplio, representativo y ajustado al dominio del corpus resulta esencial para que las sugerencias tengan sentido lingüístico y contextual. En

conjunto, ambos elementos, son determinantes para perfeccionar los procesos de corrección ortográfica y garantizar resultados confiables en etapas posteriores del análisis de texto.

Lematización

La lematización es un proceso de normalización léxica en NLP cuyo objetivo es reducir las palabras a su forma canónica o “lema”. Al aplicar lematización:

- Se eliminan variaciones flexivas (tiempo verbal, número, género).
- Se conserva el significado léxico de la palabra.
- Se mejora la coherencia del vocabulario al reducir distintas formas a un único término.

El paquete `UDPipe` implementa modelos de *Universal Dependencies* que permiten realizar tokenización, etiquetado gramatical y lematización de textos en múltiples lenguas.

```
library(udpipe)
modelo <- udpipe_download_model(language = "spanish-gsd")
udmodel <- udpipe_load_model(file = modelo$file_model)
texto <- c("Los estudiantes estaban estudiando en la biblioteca.")
anot <- udpipe_annotate(udmodel, x = texto, doc_id = 1)
```

Tabla 4: Salida de `UDPipe`: tokens, lemas y categorías gramaticales

Token	Lema	UPOS
Los	el	DET
estudiantes	estudiante	NOUN
estaban	estar	AUX
estudiando	estudiar	VERB
en	en	ADP
la	el	DET
biblioteca	biblioteca	NOUN
.	.	PUNCT

En la Tabla 4 se observan los resultados de la lematización con una oración simple. El código devuelve palabras separadas, los lemas correspondientes y el tipo de palabra procesada, entre ellas:

- DET: determinante.

- NOUN: sustantivo.
- VERB: verbo.
- ADP: adposición.
- PUNCT: puntuación.

Estos son algunos de los tipos de palabras que el paquete `UDPipe` reconoce. Este etiquetado permite filtrar aquellas categorías con mayor carga semántica (sustantivos, verbos, adjetivos, adverbios y nombres propios) y así disminuir la dimensionalidad de la entrada del modelo.

5.3.3 Representación de texto

La extracción de características relevantes (*feature extraction*) es un paso clave para el procesamiento de texto, ya que incluso el mejor modelo de *machine learning* devuelve resultados pobres si la información de entrada es inadecuada. En este apartado se desarrollarán algunas técnicas para transformar texto en una representación numérica robusta que pueda alimentar un algoritmo correctamente.

Representación vectorial

La representación vectorial de un texto consiste en la transformación de una palabra u oración a un vector de números reales dentro de un espacio vectorial de n dimensiones. Esta transformación se realiza a través de un modelo matemático y el vector resultante sirve para cuantificar las similitudes y diferencias entre dos fragmentos de texto. La manera más común de medir esta comparación es a través de la similitud coseno (o coseno del ángulo entre vectores).

Dados dos vectores, A y B , cada uno con n componentes, la similitud coseno se calcula como:

$$\cos(\theta) = \frac{A \cdot B}{|A||B|}$$

Un resultado de 1 implica similitud total (el ángulo θ es 0), y un resultado de -1 implica diferencia total (el ángulo θ es 180). Además de la similitud coseno, la distancia euclídea es otra métrica frecuente para medir la proximidad o lejanía entre vectores.

One-Hot Encoding

Una de las formas más sencillas y fundamentales de representar texto numéricamente es mediante la codificación *One-Hot*. En este enfoque, cada palabra V del corpus se asocia a un identificador único y se representa como un vector binario de dimensión $|V|$. Este vector está compuesto de todos ceros, excepto en la posición correspondiente a la palabra en el vocabulario, donde se ubica un 1.

Para ilustrar, se considera el siguiente corpus:

- D1: El perro muerde.
- D2: El gato muerde.
- D3: El perro ladra.
- D4: El gato maúlla.

Luego de preprocesar el texto, el vocabulario resultante está compuesto por cinco palabras únicas:

"perro", "muerde", "gato", "ladra", "malla"

A cada palabra de este se le asigna un índice único y será representada como un vector binario de longitud 5.

$perro \rightarrow [1, 0, 0, 0, 0]; \dots; ladra \rightarrow [0, 0, 0, 1, 0]; malla \rightarrow [0, 0, 0, 0, 1]$

Entonces, la oración D1: “El perro muerde” se representa con *One-Hot encoding* como la matriz:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

Si bien es un método simple, presenta dos desventajas principales: la alta dimensionalidad y la falta de información semántica, ya que la distancia coseno entre cualquier par de palabras distintas es 0 ya que las matrices resultan ortogonales.

Bag of words

Una de las representaciones más clásicas del texto es el enfoque *bag of words* (BoW). Este método trata a un documento como una bolsa de palabras, ignorando su orden sintáctico, pero manteniendo un registro de la frecuencia con la que aparece cada término. La intuición detrás de BoW es que la frecuencia de ciertos términos puede caracterizar el contenido de un documento.

Al igual que en *One-Hot encoding*, BoW asigna un identificador único a cada palabra del vocabulario. Luego, representa cada documento como un vector de longitud igual al tamaño del vocabulario, donde el valor en cada posición indica cuántas veces aparece esa palabra en el documento.

Tomando el ejemplo anterior, la representación BoW para la primera oración (El perro muerde) es $[1, 1, 0, 0, 0]$. Los documentos se vuelven comparables en función de su contenido léxico, como se observa en la Figura 7: los documentos D1 y D2 tienen mayor similitud ya que comparten el término *muerde*, también D1 y D3 porque comparten el término *perro*.

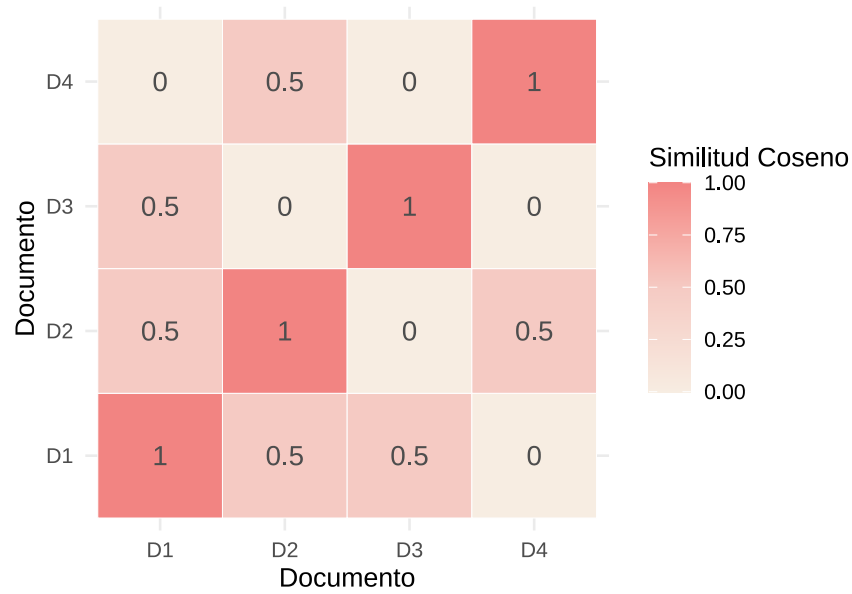


Figura 7: Similitud entre las oraciones analizadas con BoW utilizando la similitud coseno

TF-IDF

TF-IDF (*Term Frequency-Inverse Document Frequency*) es una técnica fundamental que refina a BoW al asignar pesos distintos a las palabras, destacando la importancia de aquellas que son relevantes en un documento específico pero raras en el conjunto total de documentos (corpus).

El valor TF-IDF para una palabra t en un documento d se calcula como el producto de dos componentes:

$$TF - IDF(t, d) = TF(t, d)IDF(t)$$

Donde:

- TF mide la frecuencia de un término en un documento, normalizada por la cantidad total de términos en ese documento:

$$TF(t, d) = \frac{\text{Nº ocurrencias de } t \text{ en } d}{\text{Total de términos en } d}$$

- IDF mide la rareza de una palabra a lo largo de todos los documentos. Esta métrica penaliza los términos comunes (como artículos o preposiciones) y valora los poco frecuentes, lo que aumenta su

poder discriminatorio:

$$IDF(t) = \log\left(\frac{\text{Nº total de documentos}}{\text{Nº de documentos donde aparece } t}\right)$$

Para el ejemplo de las 4 oraciones, un término como “perro” aparece en dos documentos, por lo que su IDF es más bajo que el de un término como “ladra”, que aparece solo una vez. Esto hace que “ladra” sea más “relevante” para el documento D3 que lo que es “perro” para D1. En la Tabla 5 se puede observar el resultado del cálculo detallado que realiza la técnica para las 4 oraciones.

Tabla 5: Cálculo de TF-IDF para cada documento

0.5	0.5	0.0	0	0
0.5	0.0	0.5	0	0
0.0	0.5	0.0	1	0
0.0	0.0	0.5	0	1

En comparación con BoW, TF-IDF logra una diferenciación más fina entre documentos al asignar menor peso a palabras frecuentes, lo que resulta en una matriz de similitud como la que se visualiza en la Figura 8. Los documentos D2 y D4 tienen menos similitud entre sí que D2 y D1, ya que este segundo par comparte el término *muerde* que es menos frecuente que *gato*.

Embeddings

En el campo del Procesamiento del Lenguaje Natural, los *embeddings* (incrustaciones vectoriales) han transformado la representación de texto. A diferencia de las representaciones esparsas y de alta dimensionalidad como *One-Hot* y TF-IDF, los embeddings producen representaciones vectoriales densas y compactas que capturan relaciones semánticas complejas.

Un embedding es esencialmente un vector numérico de tamaño fijo que encapsula el significado de una palabra en un contexto lingüístico. Estas representaciones permiten que palabras con significados similares se encuentren cercanas en el espacio vectorial. La base de esta técnica es la hipótesis distribucional, que establece que “el significado de una palabra está determinado por las palabras con las que co-ocurre”.

Word2Vec

Word2Vec, propuesto por Mikolov et al. (2013), marcó un hito al ofrecer una manera eficiente de generar representaciones vectoriales densas para palabras que capturan relaciones semánticas y sintácticas. *Word2Vec* utiliza redes neuronales poco profundas y ofrece dos arquitecturas principales:

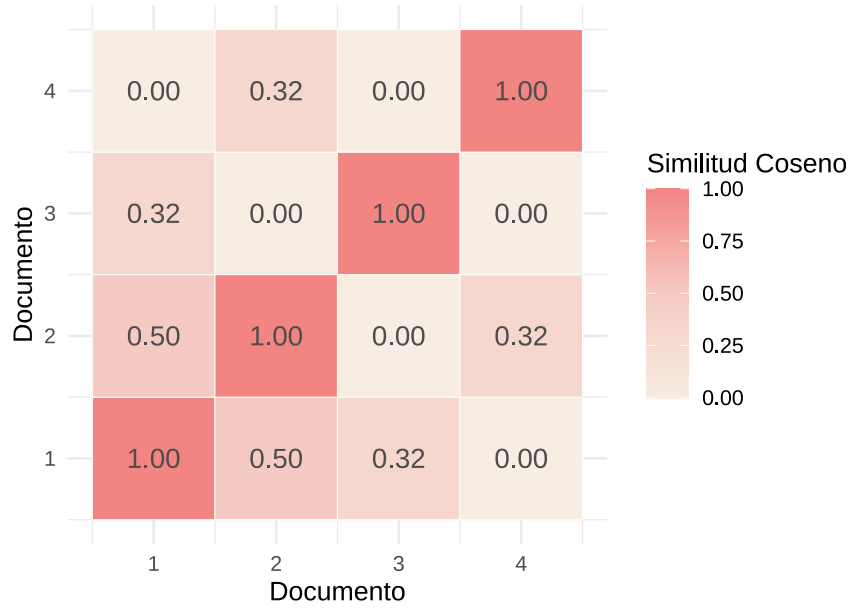


Figura 8: Similitud entre las oraciones analizadas con TF-IDF utilizando la similitud coseno

- *Continuous Bag of Words* (CBOW): Predice la palabra central (w_t) a partir del contexto circundante. Es computacionalmente eficiente y funciona bien en corpus más pequeños.
- *Skip-gram*: Predice las palabras de contexto a partir de una palabra central (w_t). Es más efectivo para capturar relaciones semánticas en corpus grandes y complejos.

Ambas arquitecturas se basan en minimizar una función de pérdida que optimiza la probabilidad de predecir correctamente palabras dentro de una “ventana de contexto”.

GloVe

GloVe (*Global Vectors for Word Representation*), presentado por Pennington et al. en 2014, surgió como un enfoque híbrido que combina las ventajas de los modelos predictivos locales (*Word2Vec*) y las estadísticas de co-ocurrencia globales del corpus.

A diferencia de *Word2Vec*, que se entrena solo con ventanas locales, *GloVe* optimiza una función de costo que relaciona directamente el producto escalar de los vectores de dos palabras ($w_i^T \tilde{w}_j$) con el logaritmo de su frecuencia de co-ocurrencia en todo el corpus ($\log X_{ij}$).

La función de costo de GloVe se define como:

$$J = \sum_{i=1}^V \sum_{j=1}^V f(X_{ij})(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

Donde X_{ij} es el número de veces que la palabra j aparece en el contexto de la palabra i , y $f(X_{ij})$ es una función de ponderación que gestiona la importancia de las co-ocurrencias. Al incorporar esta información global (la matriz de co-ocurrencia), *GloVe* logra una representación vectorial que refleja de manera más robusta la estructura semántica de todo el conjunto de datos.

BERT

Con la introducción de *BERT* (*Bidirectional Encoder Representations from Transformers*) por Devlin et al. en 2018, la representación de texto experimentó un cambio fundamental, pasando de embeddings estáticos (como *Word2Vec* y *GloVe*) a embeddings contextuales. *BERT* se basa en la arquitectura *Transformer*, la cual utiliza mecanismos de auto-atención para evaluar las dependencias entre todas las palabras de una secuencia simultáneamente.

La innovación clave de *BERT* es su capacidad de entrenamiento bidireccional. El modelo procesa la secuencia completa, permitiendo que la representación vectorial de una palabra se base en el contexto que le precede y el que le sigue. Esto resuelve la ambigüedad semántica: la palabra “banco” tendrá un vector distinto si aparece en la frase “banco del río” o “banco financiero”, ya que el modelo captura el significado específico dado por el contexto.

El modelo se preentrena sobre grandes volúmenes de texto mediante dos tareas:

- *Masked Language Model* (MLM): Se enmascara una fracción de palabras al azar y el modelo debe predecir las palabras ocultas. Esta tarea obliga a *BERT* a aprender representaciones bidireccionales profundas.
- *Next Sentence Prediction* (NSP): El modelo recibe dos oraciones y debe predecir si la segunda sigue lógicamente a la primera. Esto entrena a *BERT* a comprender las relaciones entre oraciones.

BERT produce embeddings dinámicos de alta dimensionalidad. Además de generar representaciones, el modelo es diseñado para ser ajustado (fine-tuned) con una pequeña cantidad de datos específicos de la tarea.

5.4 Latent Dirichlet allocation

La técnica de *latent Dirichlet allocation* (LDA) fue introducida por David M. Blei, Andrew Y. Ng y Michael I. Jordan (2003) como un modelo generativo probabilístico para colecciones de datos discretos, como corpus textuales. El objetivo central de LDA es descubrir automáticamente “temas latentes” dentro de un conjunto de documentos, bajo la hipótesis de que cada documento está compuesto por una mezcla de varios temas, y que cada tema está definido por una distribución de palabras.

Dado un corpus D de M documentos, donde cada documento d contiene N_d palabras (tokens) extraídas de un vocabulario de tamaño V , el modelo asume:

- Cada documento d posee una distribución de probabilidad θ_d sobre K temas, con $\theta_d \sim \text{Dirichlet}(\alpha)$.
- Cada tema k dispone de una distribución de probabilidad ϕ_k sobre el vocabulario, con $\phi_k \sim \text{Dirichlet}(\beta)$.
- Para cada palabra n en el documento d se elige un tema $z_{d,n} \sim \text{Multinomial}(\theta_d)$ y se genera la palabra $w_{d,n} \sim \text{Multinomial}(\phi_{z_{d,n}})$

De esta forma, LDA constituye un modelo jerárquico bayesiano de tres niveles: (i) distribución de temas por documento, (ii) tema seleccionado para cada palabra, (iii) palabra generada desde la distribución de dicho tema.

En la práctica, el objetivo del análisis con LDA es inferir, a partir de los datos observados, las distribuciones latentes $\{\theta_d\}_{d=1}^M$, $\{\phi_k\}_{k=1}^K$ y las asignaciones de tema $z_{d,n}$. Dado que el cálculo exacto de la distribución posterior no tiene una solución directa, se emplean métodos aproximados que serán detallados más adelante.

Una vez inferido el modelo, se pueden extraer resultados útiles como:

- para cada tema k : las palabras con mayor probabilidad en ϕ_k , lo que permite interpretar el significado de este,
- relacionar documentos entre sí, agruparlos, realizar análisis exploratorios, de clasificación, etc.

LDA se apoya en dos supuestos clave. En primer lugar, adopta la hipótesis de bag-of-words: el orden de las palabras dentro de un documento se ignora y solo importa la cantidad de veces que cada término aparece. En segundo lugar, utiliza la distribución de Dirichlet como *prior* conjugado de la distribución multinomial, lo cual permite modelar de manera natural distribuciones de probabilidad sobre conjuntos discretos y facilita la inferencia aproximada posterior.

5.4.1 LDA generativo

Sea K el número de temas y V el tamaño del vocabulario.

1. Para cada tema $k = 1, \dots, K$ se genera una distribución de palabras

$$\phi_k \sim \text{Dirichlet}(\beta)$$

donde β es un vector de hiperparámetros de dimensión V que controla la dispersión de las probabilidades de las palabras dentro de cada tema.

2. Para cada documento $d = 1, \dots, M$ se genera una distribución de temas

$$\theta_d \sim \text{Dirichlet}(\alpha)$$

donde α es un vector de hiperparámetros de dimensión K que regula cuán concentrada o dispersa es la mezcla de los temas del documento.

3. Para cada palabra $n = 1, \dots, N_d$ de un documento d , se elige un tema latente

$$z_{d,n} \sim \text{Multinomial}(\theta_d)$$

y se elige una palabra del vocabulario condicionada al tema seleccionado

$$w_{d,n} \sim \text{Multinomial}(\phi_{z_{d,n}})$$

Este procedimiento define la distribución conjunta

$$p(\Phi, \Theta, Z, W | \alpha, \beta) = \prod_{k=1}^K p(\phi_k | \beta) \prod_{d=1}^M p(\theta_d | \alpha) \prod_{n=1}^{N_d} p(z_{d,n} | \theta_d) p(w_{d,n} | \phi_{z_{d,n}})$$

Para ilustrar el funcionamiento del modelo generativo se considera un corpus simple con $K = 2$ temas y un vocabulario de cuatro palabras

$$V = \{\text{"programa"}, \text{"entrevista"}, \text{"economía"}, \text{"inflación"}\}$$

Y se definen los temas como:

- Tema 1: entretenimiento

$$\phi_1 = (0.45, 0.45, 0.05, 0.05)$$

- Tema 2: actualidad

$$\phi_2 = (0.05, 0.05, 0.40, 0.50)$$

Para generar un documento d con el modelo, el primer paso es definir una mezcla de temas, por ejemplo:

$$\theta_d = (0.7, 0.3)$$

Suponiendo que el documento tendrá $N_d = 5$ palabras, el proceso generativo produce, para cada posición n , un tema $z_{d,n}$ y luego una palabra $w_{d,n}$. De esta manera, se obtiene:

$$z_{d,n} = (1, 2, 1, 2, 1)$$

$$w_{d,n} = (\text{"entrevista"}, \text{"economia"}, \text{"entrevista"}, \text{"economia"}, \text{"programa"})$$

El documento d queda completamente generado y se continúa el proceso con $d + 1$.

5.4.2 Inferencia con LDA

El problema central de LDA es recuperar, a partir de las palabras observadas, las estructuras latentes que las generaron: las mezclas de temas por documento $\{\theta_d\}$, las distribuciones de palabras por tema $\{\phi_k\}$ y las asignaciones de tema para cada palabra $\{z_{d,n}\}$. Y, en definitiva, calcular la distribución a posterior $p(\Phi, \Theta, Z, W | \alpha, \beta)$.

Blei, Ng y Jordan (2003) demostraron que esta distribución posterior no es analíticamente resoluble debido a la dependencia que tienen las distribuciones entre sí. Esto significa que la “solución exacta” al modelo inverso es intratable. En consecuencia, LDA requiere recurrir a métodos aproximados de inferencia bayesiana como *Collapsed Gibbs Sampling* (CGS), que es el método empleado en esta tesina.

En el contexto de LDA, CGS (Griffiths & Steyvers, 2004) aparece con la idea de “colapsar” (integrar) las variables Θ y Φ , aprovechando la conjugación Dirichlet-Multinomial, y tomar muestras únicamente de las variables latentes de asignación Z . De esta manera, la muestra posterior de cada $z_{d,n}$ depende únicamente de conteo de palabras y temas observados en el resto del corpus.

$$p(z_{d,n} = k | Z_{-(d,n)}, W, \alpha, \beta) \propto (n_{d,k}^{-(d,n)} + \alpha_k) \frac{n_{k,w_{d,n}}^{-(d,n)} + \beta_{w_{d,n}}}{n_k^{-(d,n)} + \sum_v \beta_v}$$

Donde:

- $n_{d,k}^{-(d,n)}$ es la cantidad de palabras del documento d asignadas al tema k excluyendo la posición n ,
- $n_{k,w_{d,n}}^{-(d,n)}$ es la cantidad de veces que la palabra w fue asignada al tema k en todo el corpus, excluyendo la posición n ,
- $n_k^{-(d,n)}$ es el total de palabras asignadas al tema k .

Este muestreo se repite típicamente durante cientos o miles de iteraciones. La convergencia se logra cuando las asignaciones Z dejan de variar significativamente.

Una vez estimados los conteos, se reconstruyen las distribuciones posteriores mediante:

$$\hat{\theta}_{d,k} = \frac{n_{d,k} + \alpha_k}{\sum_{k'} (n_{d,k'} + \alpha_{k'})}, \hat{\phi}_{k,v} = \frac{n_{k,v} + \beta_v}{\sum_{v'} (n_{k,v'} + \beta_{v'})}$$

Estos estimadores son medias posteriores y permiten interpretar los temas y la mezcla por documento.

Para ilustrar el mecanismo de inferencia de LDA, se presenta un corpus mínimo y completamente controlado, lo que permite observar de forma explícita cómo funciona una iteración de muestreo Gibbs (Collapsed Gibbs Sampling).

- D1: perro gato.
- D2: gato maulla.
- D3: perro corre

$$V = \{\text{"perro"}, \text{"gato"}, \text{"maulla"}, \text{"corre"}\}$$

Suponiendo que existen $K = 2$ temas y que los hiperparámetros se fijan en valores simétricos ($\alpha = (1, 1)$ y $\beta =$), se comienza el procedimiento asignando un tema inicial a cada palabra del corpus (??). Luego, se definen los conteos iniciales: cuántas palabras del documento d están en el tema k , cuántas veces la palabra w se asigna al tema k y el total de palabras del tema k .

Para cada palabra $w_{d,n}$, se calcula su probabilidad a posteriori de los temas latentes empleando Gibbs. Es decir, la probabilidad de que la palabra en la posición n del documento d pertenezca al tema k , dado el resto de la información del corpus.

(ver si sumar un uso manual de gibbs para 1 palabra)

El siguiente código ajusta el modelo descrito utilizando la librería `topicmodels`, que cuenta con la función `LDA`. La Tabla 6 corresponde a la estimación posterior de θ , la mezcla de temas para cada documento. Dado que los documentos del ejemplo contienen pocas palabras y combinan términos asociados a ambos temas, las mezclas resultan uniformes (0.5, 0.5). Este comportamiento es esperable en corpus tan pequeños, donde el modelo no cuenta con suficiente evidencia para asignar un tema dominante.

La Tabla 7 corresponde a ϕ , la distribución de palabras dentro de cada tema. Aquí sí se observa una clara diferenciación: un tema asigna alta probabilidad a las palabras asociadas al léxico “gato–corre”, mientras que el otro concentra probabilidad en “perro–maulla”. Esto indica que, incluso con pocos datos,

Gibbs Sampling recupera patrones temáticos coherentes y consistentes con la estructura del corpus.

```
library(topicmodels)
library(tm)

docs <- c("perro gato",
          "gato maulla",
          "perro corre")

corpus <- Corpus(VectorSource(docs))
dtm <- DocumentTermMatrix(corpus,
                           control=list(
                             tolower=TRUE,
                             removePunctuation=TRUE,
                             stopwords=FALSE))

# LDA vía Gibbs
modelo <- topicmodels::LDA(dtm, k=2, method="Gibbs",
                           control=list(seed=123, iter=500))

# Mezcla de temas por documento (theta)
temas_doc <- round(posterior(modelo)$topics, 2)

# Distribución de palabras por tema (phi)
pal_temas <- round(posterior(modelo)$terms, 2)
```

Tabla 6: Mezcla de temas por documento

	1	2
	0.5	0.5
	0.5	0.5
	0.5	0.5

Tabla 7: Mezcla de palabras por tema

gato	perro	maulla	corre
0.62	0.03	0.03	0.32
0.03	0.62	0.32	0.03

6 Bibliografía

- Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. *Journal of Machine Learning Research*, 3, 993-1022.
- El Economista. (2024). *Streaming en Argentina: cuáles son los canales líderes y cómo monetizan sus contenidos*. <https://eleconomista.com.ar/sociedad-redes/streaming-2024-argentina-canales-lideres-figuras-estrategias-monetizacion-n74497>.
- GÉRON, A. (2019). *stringdist: Handson machine learning with scikitlearn, keras & tensorflow*. http://14.139.161.31/OddSem-0822-1122/Hands-On_Machine_Learning_with_Scikit-Learn-Keras-and-TensorFlow-2nd-Edition-Aurelien-Geron.pdf
- Kruschke, J. K. (2014). *Doing Bayesian data analysis : a tutorial with R, JAGS, and Stan*.
- Loo, M. van der. (2025). *stringdist: Approximate String Matching and String Distance Functions*. <https://cran.r-project.org/web/packages/stringdist/stringdist.pdf>
- Moyotl-Hernández, E. (2016). Método de corrección ortográfica basado en la frecuencia de las letras. *Research in Computing Science*, 124, 145-156.
- Vajjala, S., Majumder, B., Gupta, A., & Surana, H. (2020). *Practical Natural Language Processing*. O'Reilly Media.